

Course Code	ACS101
Title	Programming Languages
Course Category	Engineering Science Course (ESC)
Credits	03
NCrF Hours	45
Tools	GCC / Code::Blocks / VS Code + Google Colab / Jupyter

Main Focus of the Course Outcomes

Employability	Skill Development	Entrepreneurship
✓	✓	✓

Course Objectives:

1. Write, compile, and run programs in C and Python
2. Use variables, operators, and control flow to solve simple problems
3. Write and use functions; understand arrays and basic string operations
4. Get familiar with Python built-in data types
5. Apply Python to a small task in an emerging domain — data, automation, or IoT

Course Content

Unit	Language	Topics	Weightage
1.	C	Programming Basics (C): 1. Structure of a C program; compilation and execution 2. Data types, variables, constants, operators (arithmetic, relational, logical) 3. Input/output — scanf/printf in C 4. Type conversion; simple expressions <i>Real connect: C powers embedded systems, microcontrollers, and OS kernels.</i>	20%
2.	C	Decision Making and Control Flow (C): 1. Control Statements 2. Loops — for, while, do-while; break and continue 3. Functions — definition, call, return; call by value	20%

		4. Recursion — factorial and Fibonacci (simple examples only) <i>Real connect: Control flow is the brain of every program — from traffic signal logic to drone autopilot decision-making.</i>	
3.	C	Advanced Programming Concepts (C): 1D array — declaration, input, output, traversal - Simple operations — find max, min, sum, average - Strings in C — gets/puts, strlen, strcpy, strcmp - Pointer — concept of address and pointer variable (concept only, not deep) - Structures, - File Handling <i>Real connect: Arrays store sensor readings, image pixels, and stock prices. Pointers are the foundation of memory management in embedded systems.</i>	20%
4.	Python	Python Introduction: - Why Python — where it is used (AI, Data Science, Web, Automation) - Python basics — variables, data types, print, input, operators - Python control flow — if/else, for loop, while loop - Python functions — def, return, default arguments <i>Real connect: Python is #1 in AI/ML, Data Science, and automation — same syntax, more power, less boilerplate.</i>	20%
5.	Python	Python Data Structures and Emerging Applications - Python list, tuple, dictionary, set — basic operations only - File handling — read and write a text file; read a CSV (2–3 lines) - Introduction to NumPy — array, mean, simple arithmetic (5 lines) - Introduction to Matplotlib — plot a simple line graph (5 lines) - One mini-demo — IoT sensor simulation	20%

		<i>Real connect: Python lists power NumPy which powers TensorFlow. Matplotlib plots are used in every engineering report and data science notebook. This unit opens the door to AI, IoT, and Data Science using the same Python basics learnt in Unit IV.</i>	
--	--	---	--

Tutorial Hour — Week-by-Week Practice Problems

One tutorial hour per week. Students write and run programs. Problems below are the minimum set — teacher may add variants.

Week	Language	Tutorial Problems
1	C	1. Print your name and branch. 2. Add two numbers entered by user. 3. Find area of a rectangle.
2	C	1. Swap two numbers. 2. Check if a number is even or odd. 3. Simple calculator (+, −, ×, ÷) using if-else.
3	C	1. Find the largest of three numbers. 2. Grade calculator (marks → A/B/C/D/F). 3. Day name using switch-case.
4	C	1. Print multiplication table of N. 2. Sum of first N natural numbers using loop. 3. Count digits in a number.
5	C	1. Print prime numbers from 1 to 50. 2. Fibonacci series (first 10 terms). 3. Reverse a number.
6	C	1. Factorial using function. 2. Check palindrome number using function. 3. Find GCD of two numbers.
7	C	1. Recursive factorial. 2. Recursive Fibonacci. 3. Sum of digits using recursion.
8	C	1. Input array of N numbers; find max and min. 2. Compute average of array. 3. Linear search in array.
9	C	1. Count vowels in a string. 2. Reverse a string. 3. Check if string is palindrome.
10	C	1. Write a C program to store 5 numbers in an array and print them in reverse order. 2. Write a C program to perform basic string operations such as finding string length, copying and concatenation.
11	Python	1. Hello World in Python. 2. BMI calculator. 3. Temperature converter (°C ↔ °F). Compare C vs Python syntax.

12	Python	1. Grade calculator in Python. 2. Fibonacci using Python loop. 3. Simple calculator using functions in Python.
13	Python	1. List: marks of 5 students — max, min, average. 2. Dictionary: store name and marks of 3 students. 3. Count word frequency in a sentence.
14	Python	1. Write student records to a text file. 2. Read and display the file. 3. Read a small CSV and print rows.
15	Python	1. NumPy: create array, compute mean. 2. Matplotlib: plot a line graph of marks. 3. Mini-demo: simulate 5 IoT temperature readings and plot.

Teaching-Learning Methodology

Lectures are short and concept-focused — every idea is shown with a 3–5 line code example. Tutorial hour is compulsory hands-on coding. Students use GCC/Code::Blocks for C and Google Colab for Python (no installation needed for Python). Live coding and debugging by the teacher are part of every session. Errors are learning opportunities — never skip them.

Course Outcomes (COs)

COs	Course Outcome Statement	KL
CO1	Write, compile, and run C programs using variables, operators, and I/O	K3
CO2	Use control flow and functions in C to solve simple computational problems	K3
CO3	Work with arrays and strings in C and understand pointer concept	K2
CO4	Implement basic operations using Python built-in data structures.	K3
CO5	Apply Python to read files, plot data, and perform a simple emerging domain task	K3

CO-PO Articulation Matrix

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	1	1	3	–	–	–	–	–	–	1
CO2	3	3	2	2	3	–	–	–	–	–	–	1
CO3	3	2	1	2	3	–	–	–	–	–	–	1
CO4	3	3	2	2	3	–	–	–	–	–	–	1
CO5	3	3	2	2	3	1	1	–	–	–	–	2

Evaluation Pattern

Sr.	Component	Marks	Weightage
1.	Class Tests (CT) — short questions + write a program (C/Python)	15	15%
2.	Teacher's Assessment (TA) — Tutorial performance + Assignments + Quiz	15	15%
3.	End Semester Examination — theory + program writing (both languages)	70	70%

Suggested Textbooks

1. Problem Solving and Programming in C, Sridhar & Hariharan, Pearson
2. Basic Computer Engineering, 2ed, Sanjay Silakari, Rajesh K. Shukla, Wiley & Sons
3. Python Programming, Sridhar, Indumathi & Hariharan, Pearson
4. Let Us C – Yashavant Kanetkar (BPB) — for C basics
5. Python Crash Course – Eric Matthews (No Starch Press) — for Python units

Online Resources

1. OnlineGDB – onlinegdb.com (free browser-based C and Python compiler)
2. Google Colab – colab.research.google.com (free Python + NumPy + Matplotlib, no install)
3. HackerRank – hackerrank.com (C, Python practice, free, auto-graded)
4. NPTEL – Programming in C, IIT Bombay (free video lectures)

[Back to Course Structure & Evaluation Scheme](#)